

第二十三讲 独立实验指南

1. 要完成的任务与已知答案

本实验的核心问题是：怎样把已经观察到的数据变为一份可检查的似然优化问题？你将先算两组很小的数据，再用目标曲线、导数、数值搜索和重复采样交叉核验。只需要第13至14讲导数与偏导数、第19至20讲概率分布、第22讲抽样分布，以及前面的Python数组基础。

实验全部是小CPU合成教学，无网络、账号、GPU或付费API。第一份数据是Bernoulli序列(1,0,0,1,0,1,0,0)，第二份是Gaussian观测(0,1,2,5)，单位是任意教学单位。预期核心答案为：

- Bernoulli: $n=8$ 、 $k=3$ ，闭区间 $[0,1]$ 上唯一MLE为 $3/8$
- Gaussian: 均值2，离差平方和14，正方差MLE为3.5
- 同一Gaussian样本的无偏方差为 $14/3$ ，标准差MLE为 $\sqrt{3.5}$
- 全零Bernoulli闭区间解为0；全同值Gaussian在 $v>0$ 模型下没有有限MLE

本指南可独立跟随；完整数学证明见lecture.pdf，18题详解见answers.pdf。运行前先把上述数字手算一遍，留作参照，避免拿程序自身的输出当唯一答案。

2. 文件与环境

experiment.py提供可复用纯计算函数和命令行入口；experiment.ipynb依次提出问题、执行计算并保留图。data目录中两份CSV是手算数据，model_spec.json是模拟配置，data/README.md解释每一字段与随机流。脚本运行后才创建outputs目录，保存报告JSON和逐批估计CSV。

可从本单元目录创建独立环境：

```
conda env create -f environment.yml
conda activate ml-course-023
python experiment.py
jupyter lab experiment.ipynb
```

若使用已有Anaconda环境，应先核对Python、NumPy与Matplotlib版本，避免覆盖其他项目。建议环境不是逐平台安装承诺。核心脚本仅需Python与NumPy；Notebook还用Matplotlib、IPython和ipykernel。requirements.txt给出制作时的版本，运行说明明确未测试读者的本地安装。

在其他工作目录也能用脚本的绝对路径运行。默认输入和默认输出相对于脚本文件定位；显式指定的相对路径则相对于当前工作目录。另存结果可用：

```
python /你的路径/023/experiment.py --output /你的结果目录/mle-check
```

不要把课程正文构建所需的MathJax、PDF工具和临时缓存当作学习者运行实验的依赖。

3. 先读数据，不要跳过观察规则

打开两份CSV：表头必须恰好是`observation_id,value`，每行恰好两列；ID是唯一的ASCII标识，不能重复或带首尾空格。值是有限、无首尾空格的十进制文本。Bernoulli值仅允许0或1；脚本不自动四舍五入或把“yes”转成1。

在本单元目录执行以下代码，可用于Notebook首格之后或Python会话：

```
import experiment as e
b = e.load_csv(e.ROOT / 'data/bernoulli.csv', binary=True)
x = e.load_csv(e.ROOT / 'data/gaussian.csv')
print(b, x)
print(e.bernoulli_mle(b))
print(e.gaussian_fit(x))
```

先解释“8条记录是8次独立重启的实验”与“8行是一个结果的8次复制”的区别。脚本无法凭这几个数字检验真实独立性；本实验是在生成机制中规定iid。记录ID用于防止明显重复标识，但不同ID也可能来自同一随机事件。

4. 比较完整似然与计数似然

在 $p=0.25$ 、 0.375 、 0.5 处分别调用 `bernoulli_loglik(b,p)`，与手算自然对数比较。再调用带 `count_observation=True` 的版本，检查两种log似然之差总为 $\log(56)$ 。不要先对 n 个小概率做长连乘。

Notebook第一个图同时画原序列似然与log似然。检查横轴是候选参数 p ，数据本身没有随曲线点改变。图中的最大位置应接近 0.375 ；网格含不含 0.375 只影响绘图近似，解析MLE不由网格定义。

接着调用 `bernoulli_derivatives(b,0.375)`：一阶导数应为0，二阶导数约-34.133333。用中心差分 $(\ell(p+h) - \ell(p-h))/(2h)$ 在 $p=0.3$ 处与解析导数比较。尝试 h 为0.01、0.001、0.0001、0.00001，解释大步长的截断误差与小步长的浮点消去为何不同。

5. 数值搜索与边界分支

调用 `bisect_bernoulli(b)`，主样本从 $[0,1]$ 出发三次中点评价就恰好找到 0.375 。输出中的`bracket`是最后保留区间，`converged`表示区间已小于容差。程序预先处理 $k=0$ 与 $k=n$ ；导数函数本身只接受远离0和1的内部参数。

把数据另存为零列表，分别检查MLE、 $p=0$ 处的log似然，以及 $p=0$ 但样本包含1时的log似然。前者应是0，后者是负无穷。二者都不应成为NaN，也不应被自动裁剪成很小的正概率。

Notebook第二个图对照 $k=0$ 、3、8的相对似然。随后查看logit变式：有限 t 对应开区间概率，全零样本只有 t 趋近负无穷时才接近上确界。程序没有实现“迭代到非常负就宣称找到MLE”的伪成功流程。

6. Gaussian的均值与方差要同时核验

手写主数据的离差(-2,-1,0,3)，再读 `gaussian_fit(x)`。把均值2、方差3.5代入 `gaussian_derivatives`，两个偏导数均应为0。再检查 `gaussian_loglik(x,2,3.5)` 高于把均值或方差单独移开的候选。

Notebook第三图画二维目标等高线。第四图比较正确剖面与漏归一化项的错误目标，并展示全同值数据的无界增长。参数 v 始终是方差，不是标准差。若给正态随机生成器传入 v 而非 $\sqrt{v}$ ，就会把设定的方差错误平方。

用黄金分割搜索核对方差：

```
import math
fit = e.gaussian_fit(x)
sol = e.golden_maximize(
    lambda z: e.gaussian_loglik(x, fit['mean'], math.exp(z)),
    -5, 5)
print(sol, math.exp(sol['x']))
```

最优 z 应接近 $\log(3.5)$ ，指数变换后的方差在 $1e-6$ 量级相对/绝对容差内与3.5一致。搜索假设目标连续单峰，正文已证明 $S>0$ 时 $\log$ 方差方向严格凹；通用函数的`converged`标记本身不能证明任意其他目标的全局最优。

7. 改一个观测，再看一个失败输入

把最后一项由5改成21。均值应从2变为6，方差MLE从3.5变为75.5。Notebook第五图连续改变最后一项 t ，帮助看出均值线性、方差二次变化。不要在看到这种变化后自动删除极端点；先说明它是错误记录还是你想研究的真实尾部。

再输入(7,7,7,7)或单个值7。`gaussian_fit`应返回`no_finite_mle_unbounded_likelihood`，`variance_mle`为None。均值字段只是描述这批数据的均值，不能与None凑成一个合法联合MLE。若有人声称“把 v 加一个极小量即可修好”，请指出这增加了方差下界约束，改变了问题。

8. 重复4000批：一次拟合与平均偏差分开

默认配置为Bernoulli参数3/8，Gaussian均值2、方差3.5； n 取4、16、64，每个 n 重复 $R=4000$ 批。调用 `repeat_sampling(e.load_spec())` 得到`summaries`与`samples`。`samples[n]`的每个向量有4000项，一项对应一批完整数据的估计；原始随机矩阵形状为 (R,n) 。

理论上Bernoulli比例MLE与Gaussian均值MLE无偏；Gaussian方差MLE期望为 $(n-1)3.5/n$ ，无偏方差版本期望3.5。 $n=4、16、64$ 时MLE期望依次为2.625、3.28125、3.4453125。默认实际 $n=4$ 两种方差估计均值约2.644831和3.526442，不必与理论逐位相等。

Notebook第六图把解析期望曲线与模拟点放在一起。解释R变大与n变大的不同效果。修改实验时另存JSON，使用脚本的--spec参数；Notebook首格锁定已讲解的默认配置，发现改动会明确停止，防止问题还写旧参数。有限模拟只是展示机制，不能作为现实性能或理论证明。

9. 接口契约与明确拒绝

数值函数接受Python整数/浮点或不超过64位精度的NumPy实数标量，拒绝bool、文本、复数、对象dtype、扩展精度、非有限数和二维数组。列表逐元素核验后才转float64；如果调用者先把混合数据强制转为纯数值数组，已经丢失的类型来源无法追溯。CSV文本在专用解析层按十进制语法转换，不意味着数值API也接受任意文本。

向量长度为1至100000，数值绝对值至多 $1e6$ ，非零输入绝对值至少 $1e-12$ 。Gaussian评价方差与非退化拟合方差限制在 $[1e-12, 1e12]$ ；数学模型仍为 $v > 0$ ，数值范围是保守实现限制，超范围报错而非裁剪。内部平方、求和、结果有限性也检查；极小差异导致的方差不受支持时明确失败。

模拟配置必须恰好具备既定键；样本量为2至512、最多8个严格递增不同值，R为2至10000，每个分布的R乘样本量总和不超过二百万。模拟均值限 $[-100, 100]$ 、方差限 $[1e-4, 10000]$ 。完整配置在创建随机流前检查；黄金分割的所有输入参数也在调用目标函数前检查。

依次尝试 `[0, True]`、`[0, '1']`、`[0, float('nan')]`、`[[0, 1]]`、Gaussian方差0。这些都应得到明确ValueError。输入无效时，不应靠删值、隐式类型转换或关闭检查来“让实验跑通”。

10. 验收与故障排查

先运行 `reference_checks()` 和 `boundary_checks()`。这些检查使用显式require，不依赖会被python -O移除的assert。再从不同工作目录各启动一个新进程，比较报告与CSV；用python -O重复。Notebook要重启内核并按顺序全部执行，不能凭单格输出成功就认为无状态依赖。

为验证旧输出保护，先保留正确运行的两份结果；另存一份包含重复ID或nan的CSV，再用--gaussian指向坏文件且输出仍指向旧目录。要求进程失败，旧结果字节完全不变。脚本在输入、计算和序列化完成后才写文件，因此坏输入不会覆盖旧输出；它不提供突然断电、磁盘写满时的多文件原子事务保证。

若import失败，检查环境和文件路径；若None出现，先看是否全同值导致无有限MLE；若数值范围错误，查看单位、数据尺度和方差是否超界。不要把任何错误一律解释为“优化器不稳定”。完成后用自己的话写三段：一个正确推导、一个边界例子、一个数值失败例子。

保留Notebook已在新Python进程的真实InProcessKernel执行并保存PNG。浏览器Jupyter界面、常规socket传输、不同操作系统安装和跨NumPy版本逐位复现没有测试；这些边界在verification.json与README.md中明确记录。